

Shellsort, also known as **Shell sort** or **Shell's method**, is an in-place comparison sort. It can be understood as either a generalization of sorting by exchange (bubble sort) or sorting by insertion (insertion sort).^[3] The method starts by sorting pairs of elements far apart from each other, then progressively reducing the gap between elements to be compared. By starting with far-apart elements, it can move some out-of-place elements into the position faster than a simple nearest-neighbor exchange. The running time of Shellsort is heavily dependent on the gap sequence it uses. For many practical variants, determining their time complexity remains an open problem.

The algorithm was first published by Donald Shell in 1959, and has nothing to do with shells.^{[4][5]}

Description

Shellsort is an optimization of insertion sort that allows the exchange of items that are far apart. The idea is to arrange the list of elements so that, starting anywhere, taking every h th element produces a sorted list. Such a list is said to be h -sorted. It can also be thought of as h interleaved lists, each individually sorted.^[6] Beginning with large values of h allows elements to move long distances in the original list, reducing large amounts of disorder quickly, and leaving less work for smaller h -sort steps to do.^[7] If the list is then k -sorted for some smaller integer k , then the list remains h -sorted. A final sort with $h = 1$ ensures the list is fully sorted at the end,^[6] but a judiciously chosen decreasing sequence of h values leaves very little work for this final pass to do.

In simplistic terms, this means if we have an array of 1024 numbers, our first gap (h) could be 512. We then run through the list comparing each element in the first half to the element in the second half. Our second gap (k) is 256, which breaks the array into four sections (starting at 0, 256, 512, 768), and we make sure the first items in each section are sorted relative to each other, then the second item in each section, and so on. In practice the gap sequence could be anything, but the last gap is always 1 to finish the sort (effectively finishing with an ordinary insertion sort).

An example run of Shellsort with gaps 5, 3 and 1 is shown below.

	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}	a_{12}
Input data	62	83	18	53	07	17	95	86	47	69	25	28
After 5-sorting	17	28	18	47	07	25	83	86	53	69	62	95
After 3-sorting	17	07	18	47	28	25	69	62	53	83	86	95
After 1-sorting	07	17	18	25	28	47	53	62	69	83	86	95

The first pass, 5-sorting, performs insertion sort on five separate subarrays (a_1, a_6, a_{11}) , (a_2, a_7, a_{12}) , (a_3, a_8) , (a_4, a_9) , (a_5, a_{10}) . For instance, it changes the subarray (a_1, a_6, a_{11}) from $(62, 17, 25)$ to $(17, 25, 62)$. The next pass, 3-sorting, performs insertion sort on the three subarrays (a_1, a_4, a_7, a_{10}) , (a_2, a_5, a_8, a_{11}) , (a_3, a_6, a_9, a_{12}) . The last pass, 1-sorting, is an ordinary insertion sort of the entire array $(a_1, \dots, a_{12})$.

As the example illustrates, the subarrays that Shellsort operates on are initially short; later they are longer but almost ordered. In both cases insertion sort works efficiently.

Unlike insertion sort, Shellsort is not a stable sort since gapped insertions transport equal elements past

one another and thus lose their original order. It is an adaptive sorting algorithm in that it executes faster when the input is partially sorted.

Pseudocode

Using Marcin Ciura's gap sequence, with an inner insertion sort.

```
# Sort an array a[0...n-1].
gaps = [701, 301, 132, 57, 23, 10, 4, 1] # Ciura gap sequence

# Start with the largest gap and work down to a gap of 1
# similar to insertion sort but instead of 1, gap is being used in each step
foreach (gap in gaps)
{
    # Do a gapped insertion sort for every element in gaps
    # Each loop leaves a[0..gap-1] in gapped order
    for (i = gap; i < n; i += 1)
    {
        # save a[i] in temp and make a hole at position i
        temp = a[i]
        # shift earlier gap-sorted elements up until the correct location for a[i] is found
        for (j = i; (j >= gap) && (a[j - gap] > temp); j -= gap)
        {
            a[j] = a[j - gap]
        }
        # put temp (the original a[i]) in its correct location
        a[j] = temp
    }
}
```

Gap sequences

The question of deciding which gap sequence to use is difficult. Every gap sequence that contains 1 yields a correct sort (as this makes the final pass an ordinary insertion sort); however, the properties of thus obtained versions of Shellsort may be very different. Too few gaps slows down the passes, and too many gaps produces an overhead.

The table below compares most proposed gap sequences published so far. Some of them have decreasing elements that depend on the size of the sorted array (N). Others are increasing infinite sequences, whose elements less than N should be used in reverse order.